

Software Architecture and Design Specification

Online Bookstore System - Buying & Selling Platform

Field	Details
Project	Online Bookstore System
Version	1.0
Date	02-10-2026
Status	Draft
Team	Team 4
Authors	Pruthviraj H (PES2UG24CS381) Puneet Lingashettar (PES2UG24CS382) Punith K (PES2UG24CS383) Purvitha Machireddy (PES2UG24CS384)

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Team 4	Initial draft of architecture and design

Approvals

Role	Name	Signature/Date
Instructor		
Team Lead		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Bookstore System, a platform where customers search and buy new or pre-owned books, sellers submit books for sale, and administrators manage the catalog, inventory, orders, reviews and sales data.

1.2 Scope

Covers user authentication and profiles, catalog search, cart and wishlist, seller listings, checkout and payment, order management, admin functions, and the security, performance and accessibility requirements in the SRS. Physical warehousing, courier operations and raw card processing are out of scope.

1.3 Audience

Developers, QA engineers, security auditors, instructors and maintenance teams.

1.4 Definitions

Term	Meaning
JWT	JSON Web Token for stateless authentication
RBAC	Role-Based Access Control
DRF	Django REST Framework
PCI-DSS	Payment Card Industry Data Security Standard
TLS	Transport Layer Security
ACID	Atomicity, Consistency, Isolation, Durability
ADR	Architecture Decision Record

2. Document Overview

2.1 How to use this document

Provides the architectural deliverables: component and sequence diagrams, ADRs, the STRIDE threat model and the API design.

2.2 Related Documents

SRS, Software Test Plan (STP), Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals: secure handling of accounts and payments; 95% of catalog/detail API calls under 2 s; at least 500 concurrent sessions; 99.5% monthly uptime; consistent inventory under concurrent purchases; accessible, responsive UI (Lighthouse above 90).

Constraints: payments only via a PCI-DSS-compliant gateway (Stripe/Razorpay) with tokenization; stack fixed to React, Django REST Framework and PostgreSQL 15+; HTTPS with TLS 1.2+ everywhere; Python 3.11+; Docker deployment on AWS EC2 or Render.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Customers	Easy search, safe payment, order tracking, privacy
Sellers	Simple listing flow, approval status, stock accuracy
Administrators	Catalog, inventory, moderation, sales insight
Instructors / Auditors	Traceability, security, documentation quality
Developers / QA	Modularity, testability, clear APIs

3.3 Component (UML) Diagram

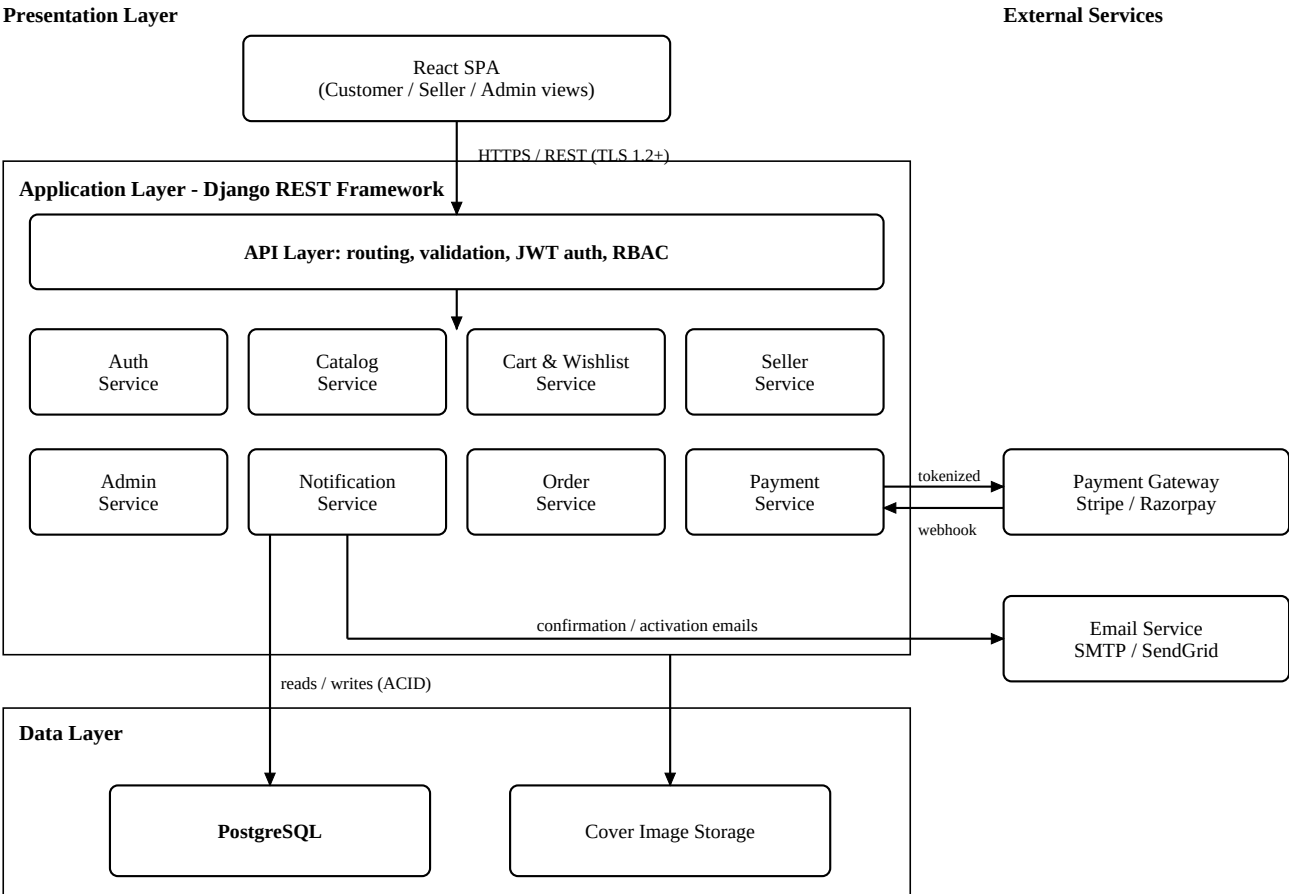


Figure 1: Layered component view of the Online Bookstore System.

3.4 Component Descriptions

Component	Responsibility
React SPA	Responsive UI for all roles; calls backend over HTTPS; keeps access token in memory
API Layer	Routing, request validation, JWT verification, RBAC checks, standard error format
Auth Service	Registration, email activation, login, password reset, profile and address management, token revocation on logout
Catalog Service	Search (title, author, ISBN, genre), filters (price, condition, rating), sorting, book details
Cart & Wishlist Service	Cart CRUD, subtotal and tax calculation, wishlist persistence, move-to-cart
Seller Service	Book listing with condition (New, Like New, Good, Acceptable), image upload and validation, Pending Approval state
Order Service	Checkout, order creation, status lifecycle (Placed, Processing, Shipped, Delivered, Cancelled), history, invoice data

Component	Responsibility
Payment Service	Creates gateway payment intents, handles webhooks, stores transaction ID only
Admin Service	Catalog management, listing approval, stock updates, low-stock flag (below 5), review moderation, sales dashboard
Notification Service	Activation, reset and order confirmation emails
PostgreSQL	Users, books, carts, orders, inventory, reviews, payment audit records

3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered three-tier architecture (React client, Django REST application layer, PostgreSQL data layer), with the application layer organised into modular Django apps by domain.

Rationale: clear separation of concerns, a good fit for a four-member team, simple deployment in one Docker image, and ACID transactions in a single database for order and inventory consistency.

Rejected: Microservices - extra operational complexity and distributed transactions are unnecessary at this scale.

Architecture Decision Records (summary)

ID	Decision	Reason
ADR-01	Layered modular monolith	Simplicity, team size, transactional integrity
ADR-02	JWT with 15-minute access tokens and server-side revocation	Stateless scaling with logout safety (BOOK-SR-003)
ADR-03	Gateway-hosted tokenized payments	Keeps PAN/CVV out of our system (BOOK-SR-005)
ADR-04	Row-level locking on stock during checkout	Last-copy concurrent purchase has one winner (BOOK-NF-006)

3.6 Technology Stack & Data Stores

Layer	Technology
Frontend	React.js, Axios/Fetch
Backend	Python 3.11+, Django, Django REST Framework, SimpleJWT
Database	PostgreSQL 15+
Payments	Stripe or Razorpay
Email	SMTP / SendGrid
Deployment	Docker, Linux, AWS EC2 or Render
Testing	Postman, JMeter, Lighthouse, Browser DevTools
Version control	Git

3.7 Risks & Mitigations

Risk	Mitigation
Overselling the last copy	SELECT ... FOR UPDATE inside an atomic transaction; out-of-stock response for the loser
Payment succeeded but order not created	Idempotent webhook handler; reconcile on gateway callback
Slow search at 10,000+ records	DB indexes on title, author, ISBN, genre; pagination

Risk	Mitigation
Token theft	Short expiry, HTTPS only, revocation list
Gateway or email outage	Retry queue for emails; clear user error for payment failure
Load spikes (500 sessions)	Stateless API, horizontal scaling, JMeter validation

3.8 Traceability to Requirements

Requirement	Component
BOOK-F-001 to F-005 (auth, profile, roles)	Auth Service
BOOK-F-006 to F-010 (search, catalog, cart, wishlist)	Catalog, Cart & Wishlist Services
BOOK-F-011 (seller listing)	Seller Service
BOOK-F-012 to F-016 (checkout, payment, orders, email)	Order, Payment, Notification Services
BOOK-F-017 to F-020 (admin, inventory, reviews, analytics)	Admin Service
BOOK-NF-001 / NF-003 (performance)	API Layer, DB indexes
BOOK-NF-002 (uptime)	Deployment, health checks
BOOK-NF-004 (accessibility)	React SPA
BOOK-NF-006 (ACID)	Order Service, PostgreSQL
BOOK-SR-001 to SR-005	Auth Service, API Layer, Payment Service

3.9 Security Architecture

Threat modeling (STRIDE):

Threat	Mitigation
Spoofing	Email-verified accounts, hashed passwords (PBKDF2/bcrypt), signed JWTs
Tampering	HTTPS/TLS 1.2+, server-side validation, signed gateway webhooks
Repudiation	Audit records for orders, payments and admin actions
Information disclosure	No plain-text passwords, no PAN/CVV stored, no secrets in logs
Denial of service	Rate limiting, pagination, load testing
Elevation of privilege	Server-side RBAC; customers get 403 on seller/admin APIs (e.g. /api/admin/sales/)

4. Design

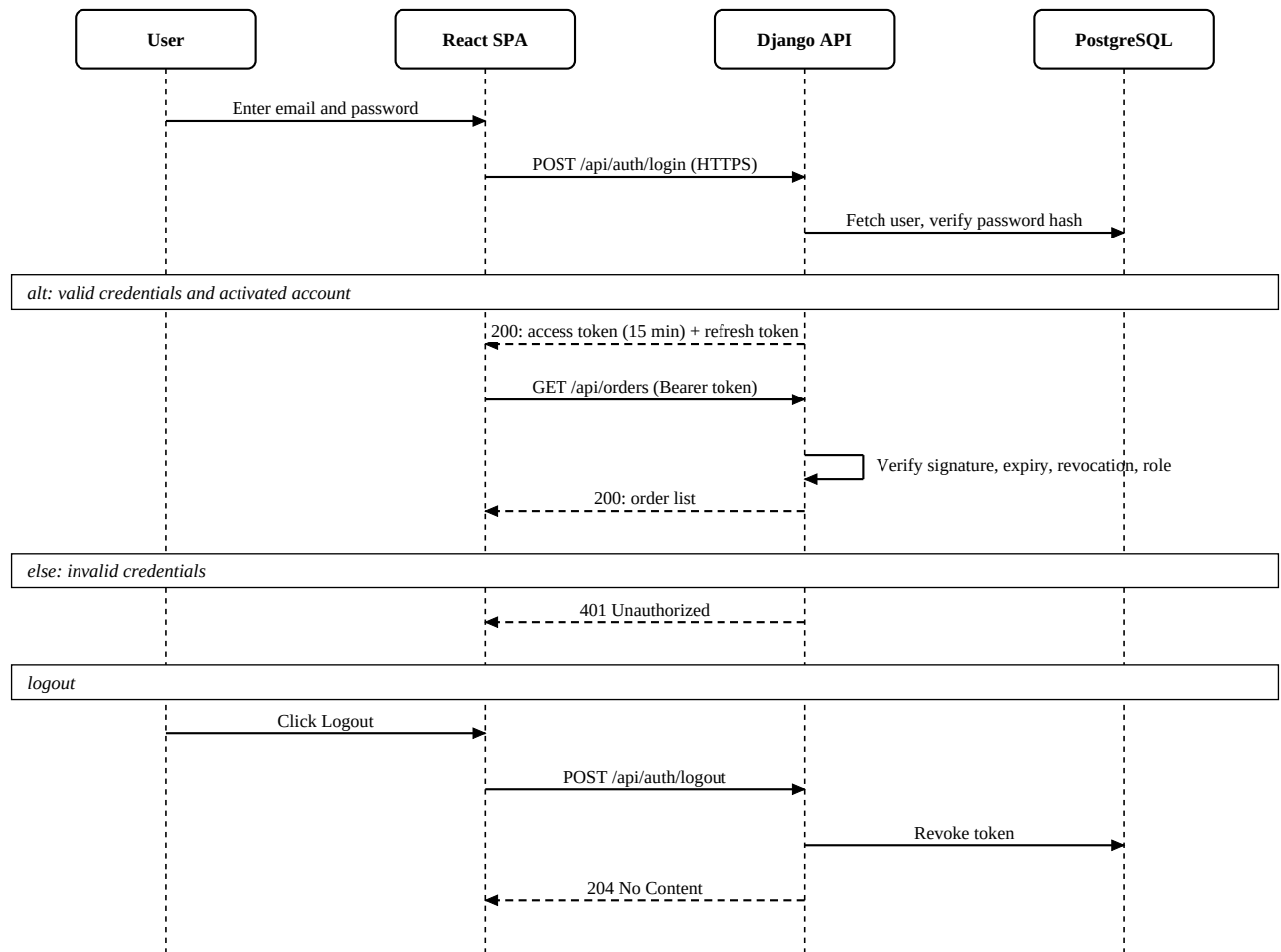
4.1 Design Overview

The backend is split into Django apps (accounts, catalog, cart, sellers, orders, payments, adminpanel, notifications), each exposing REST endpoints through DRF serializers and viewsets. Business rules live in service functions; views stay thin. Permissions are enforced by DRF permission classes per role.

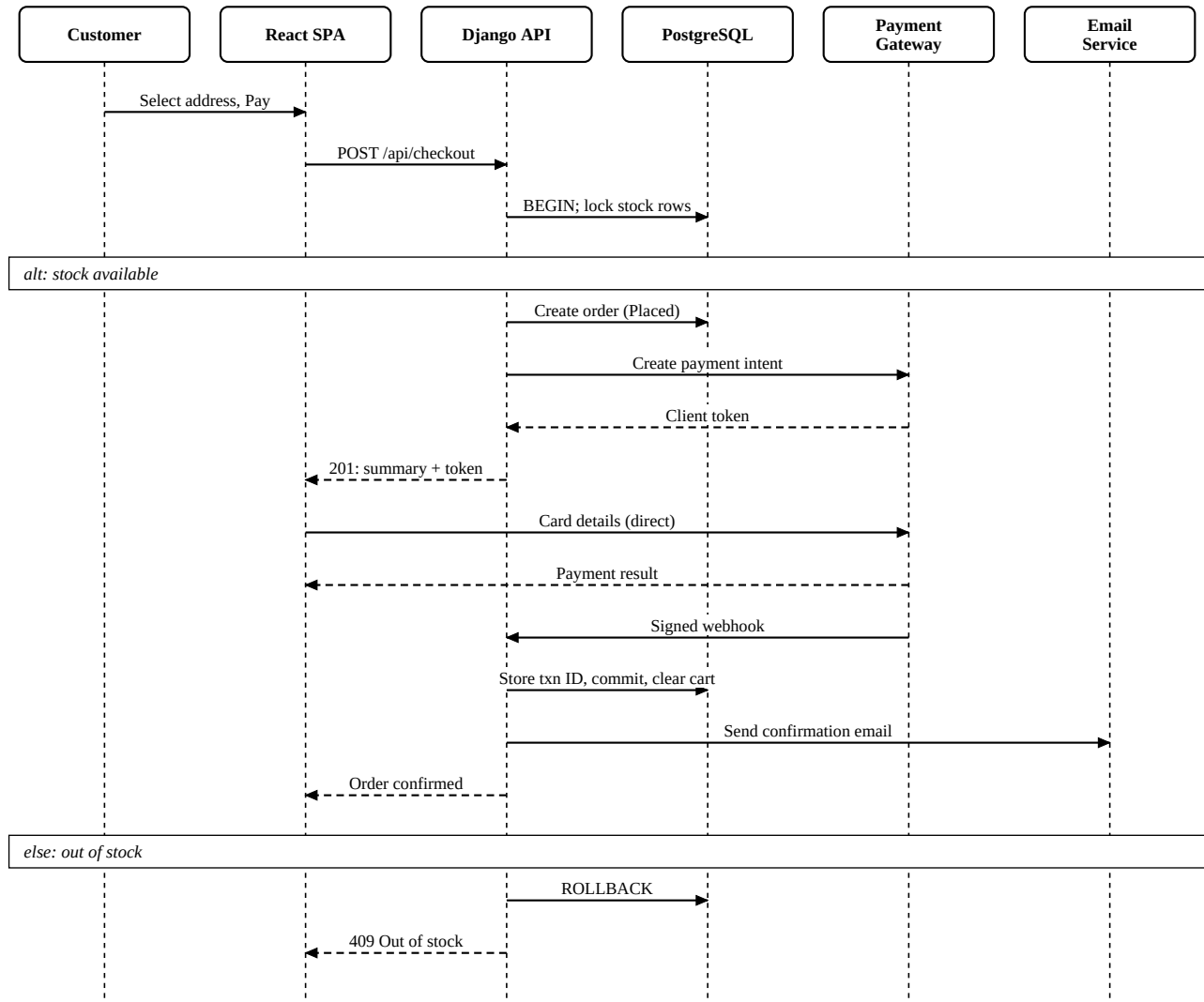
Key entities: User, Address, Book, Cart, CartItem, Wishlist, Order, OrderItem, Payment (transaction ID and status only), Review, StockLog.

4.2 UML Sequence Diagrams

Flow 1 - Login and JWT-protected access



Flow 2 - Checkout and payment



4.3 API Design

All endpoints use HTTPS and JSON. Protected endpoints require Authorization: Bearer <access_token>.

Component 1 - Auth Service

Item	Detail
Endpoint	/api/auth/login
Method	POST
Request	{email, password}
Response (200)	{access, refresh, role}
Errors	400 Missing fields, 401 Invalid credentials or inactive account

Component 2 - Order Service

Item	Detail
Endpoint	/api/checkout
Method	POST
Auth	Customer

Item	Detail
Request	{addressId, cartId}
Response (201)	{orderId, items, subtotal, tax, shipping, total, paymentToken}
Errors	401 Unauthenticated, 409 Out of stock, 422 Invalid address

Component 3 - Catalog Service

Item	Detail
Endpoint	/api/books?q=&genre=&condition=&minPrice=&maxPrice=&rating=&sort=
Method	GET
Response (200)	{count, results: [{id, title, author, isbn, price, condition, rating}]}
Errors	404 on /api/books/{id} with an invalid ID

Component 4 - Admin Service

Item	Detail
Endpoint	/api/admin/sales/
Method	GET
Auth	Admin only
Response (200)	{totalSales, ordersByStatus, topBooks, lowStock}
Errors	401 Unauthenticated, 403 Non-admin role

Other main endpoints: /api/auth/register, /api/auth/activate/{token}, /api/auth/password-reset, /api/cart, /api/wishlist, /api/seller/books, /api/orders/{id}, /api/payments/webhook, /api/admin/books, /api/admin/reviews.

4.4 Error Handling, Logging & Monitoring

- **Standard error format:** {"error": {"code": "...", "message": "..."}} with correct HTTP codes (400, 401, 403, 404, 409, 422, 500).
- **Logging:** structured logs with request ID; passwords, tokens and payment data are never logged.
- **Audit trail:** order status changes, payment events and admin actions.
- **Monitoring:** health-check endpoint for uptime (99.5% target), API response times (p95 under 2 s), payment and email failure rates, low-stock alerts.

4.5 UX Design

- Responsive layouts for 1920x1080, 768x1024 and 375x812.
- WCAG 2.1 AA: keyboard navigation, visible focus, sufficient contrast, alt text on cover images, labelled forms.
- Clear status messages for payment, out-of-stock and Pending Approval states.
- Role-specific navigation for Customer, Seller and Admin.

4.6 Open Issues & Next Steps

- Finalise gateway choice (Stripe vs Razorpay).
- Decide token revocation storage (DB table vs cache).
- Confirm data retention period for BOOK-NF-005.
- Future enhancements: recommendations, seller payouts, mobile app, wishlist price alerts.

5. Appendices

5.1 Glossary

JWT, RBAC, DRF, PCI-DSS, TLS, ACID, ADR, PAN, CVV, WCAG, RTM.

5.2 References

IEEE 42010, OWASP ASVS, NIST SP 800-160, Online Bookstore SRS, Team 4 Software Test Plan.

5.3 Tools

PlantUML, draw.io, Mermaid, Swagger/OpenAPI, Postman, JMeter, Lighthouse.